# Assignment 12

This exercise sheet is about convergence properties of one-step methods and solvability for implicit Runge–Kutta methods (last exercise).

---

## Graded exercise

---

**Exercise 39** (Periodic fishing)

We model the temporal behavior of a fish population $u(t)$ with the *logistic differential equation*

$$u'(t) = (A - H(t))u(t) - Bu(t)^2, \tag{1}$$

with parameters $A, B > 0$. The function $H = H(t)$ models fishing: $H > 0$ when fishing occurs, while $H = 0$ means that no fishing is taking place. In a realistic model, $H$ is a jump function, with fishing periodically activated and deactivated. We aim at solving (1) numerically using *Heun's method*. For a generic ODE $u'(t) = f(t, u)$, Heun's method reads:

$$\begin{aligned} u_n^* &= u_{n-1} + \tau f(t_{n-1}, u_{n-1}), \\ u_n &= u_{n-1} + \frac{\tau}{2}\left(f(t_{n-1}, u_{n-1}) + f(t_n, u_n^*)\right). \end{aligned}$$

It is an example of a so-called *predictor-corrector* method, with explicit Euler as predictor and Crank-Nicolson as corrector.

**(a)** Write a PYTHON function `Heun(f,t0,u0,tau,N)` that, given in input a function handle `f` to an ODE right-hand side, the initial time `t0` and state `u0`, the time step size `tau` and the number `N` of time steps, solves $u'(t) = f(t, u), t > t_0$ with $u(t_0) = u_0$ using Heun's method. The routine should return the solution at all time steps, namely $u_0, \ldots, u_N$. For generality, write the routine in such a way that it works for a system of ODEs, that is in the case that $u : [t_0, \infty) \to \mathbb{R}^m$ with $m$ possibly greater than one.

*Hint*: Note that this routine can be written very similarly to those implemented for the previous assignment.

**(b)** Write a PYTHON program which solves numerically the differential equation (1) on the time interval $[0, 9]$ using Heun's method.

Set the parameters to $A = B = 1$, the initial value to $u(0) = 2$ and the switching function to

$$H(t) = \begin{cases} 0.2, & 0 \le t \le 3, \\ 0, & 3 < t < 6, \\ 0.2, & 6 \le t \le 9. \end{cases} \tag{2}$$

Use the time step size $\tau = \frac{1}{8}$ and produce a plot of the solution over time (don't forget to hand in the plot!).

*Hint*: Since in task (a) you were asked to implement Heun's method for a system of ODEs, here, that the ODE is scalar, don't forget to convert your input $u(0)$ to the correct data type. In `numpy`, the function `numpy.atleast_1d` can be useful (but you don't need to use it, this is just a suggestion).

*Hint*: You don't have to, but it might be easier, also for the next tasks, to use a proper PYTHON function (instead of e.g. a lambda function) to implement the switching function (2).

**(c)** Write a PYTHON script that, for the time step sizes $\tau = 2^{-\ell}$, $\ell = 1, 2, \ldots, 6$, computes the solution of (1) with the same parameters as in the previous task and the maximum relative error at the grid points $\max_{n=0,\ldots,N} |u(t_n) - u_n|/|u(t_n)|$. Use this script to fill in a table reporting the maximum relative error for $\tau = 2^{-\ell}$, $\ell = 1, 2, \ldots, 6$ (to be handed in). Do you observe the convergence order $p = 2$ (expected from Heun's method)?

*Hint*: To compute the error, you can use the exact solution for this setting, given by

$$u(t) = \begin{cases} 4/(5 - 3e^{-0.8t}), & 0 \le t \le 3, \\ 4/(4 + e^{3-t} - 3e^{0.6-t}), & 3 < t < 6, \\ 4/(5 - e^{4.8-0.8t} + e^{1.8-0.8t} - 3e^{-0.6-0.8t}), & 6 \le t \le 9. \end{cases}$$

*Hint*: Since the chosen time step sizes produce discrete times that capture the discontinuities of the function $H$, we want to take advantage of this. Therefore, when evaluating the function $H$ at the discontinuity points, we take the value from the left or from the right depending on the time step. This means that, when performing the step from $t_{n-1}$ to $t_n$, we evaluate $H(t_{n-1})$ by taking the value from the *right*, and $H(t_n)$ by taking the value from the *left*, so that, within a time step, the two values coincide for the time step sizes that you need to test in this task. Programming-wise, this can be achieved for example by using a flag when defining the function $H$ and by specifying the flag in Heun's method when evaluating the right-hand side $f$. You are therefore allowed here to slightly modify your implementation for (2) in (b) and the routine for Heun's method from (a). It is sufficient that you hand in these modified versions as long as the routines still work correctly also for tasks (a) and (b). **Note**: Task (d) on next page!

**(d)** Rerun the code from the previous task but now using the time step sizes $\tau = 1.1 \times 2^{-\ell}$, $\ell = 1, 2, \ldots, 6$. Again, fill in and hand in a table of the maximum relative error for each time step. Do you observe convergence order $p = 2$ in this case? How do you explain this?

---

## Other exercises

---

**Exercise 40** (Order of consistency and Butcher tableau)
We consider the following one-step method (using the same notation as in the lecture):

$$u_n = u_{n-1} + \frac{2}{9} f(t_{n-1}, u_{n-1}) + \frac{1}{3} f\left(t_{n-1} + \frac{\tau}{2}, u_{n-1} + \frac{\tau}{2} f(t_{n-1}, u_{n-1})\right) + \frac{4}{9} f\left(t_{n-1} + \frac{3}{4}\tau, u_{n-1} + \frac{3}{4}\tau f\left(t_{n-1} + \frac{\tau}{2}, u_{n-1} + \frac{\tau}{2} f(t_{n-1}, u_{n-1})\right)\right).$$

**(a)** Show by direct calculation (namely, without just verifying the order conditions) that the method above has consistency order at least $p = 3$.

**(b)** The method described is actually a Runge-Kutta method. Write down its Butcher tableau.

**Exercise 41** (Order from the Butcher tableau)
For the solution of this exercise, we report the order conditions of Runge–Kutta methods for convenience:

| Order $p$ | Order conditions | |
|---|---|---|
| 1 | $\sum_i b_i = 1$ | |
| 2 | $\sum_i b_i c_i = 1/2$ | |
| 3 | $\sum_i b_i c_i^2 = 1/3$ | $\sum_{i,j} b_i a_{ij} c_j = 1/6$ |
| 4 | $\sum_i b_i c_i^3 = 1/4$ | $\sum_{i,j} b_i c_i a_{ij} c_j = 1/8$ |
| | $\sum_{i,j} b_i a_{ij} c_j^2 = 1/12$ | $\sum_{i,j,k} b_i a_{ij} a_{jk} c_k = 1/24$ |

**(a)** We consider the following Butcher tableaus for two explicit Runge–Kutta methods with $s = 3$ stages:

$(i)$

| | | | |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| $\frac{1}{3}$ | $\frac{1}{3}$ | 0 | 0 |
| $\frac{2}{3}$ | 0 | $\frac{2}{3}$ | 0 |
| | $\frac{1}{4}$ | 0 | $\frac{3}{4}$ |

$(ii)$

| | | | |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 1 | 0 | 0 |
| $\frac{1}{2}$ | $\frac{1}{3}$ | $\frac{1}{6}$ | 0 |
| | $\frac{1}{6}$ | $\frac{1}{6}$ | $\frac{2}{3}$ |

Which is the convergence order of each of these schemes?

**(b)** We consider the following Butcher scheme:

$$\begin{array}{c|ccc} 0 & 0 & 0 & 0 \\ c_2 & a_{21} & 0 & 0 \\ \frac{1}{2} & a_{31} & a_{32} & 0 \\ \hline & \frac{1}{6} & \frac{1}{6} & b_3 \end{array}$$

How should the parameters $c_2, a_{21}, a_{31}, a_{32}$ and $b_3$ be chosen, so that the method has order 2? Is it possible to choose the parameters such that the method has order 3 or 4?

**Exercise 42** (Solvability for implicit Runge–Kutta methods)

When taking a step from $(t_{n-1}, u_{n-1})$ to $(t_n, u_n)$ with an implicit Runge-Kutta method, the intermediate vectors $k_i = k_{n,i} \in \mathbb{R}^m$ must be computed by solving a nonlinear system, see equation (7.9a) in the lecture notes. In this exercise, we show that the latter has a unique solution. We assume that the ODE's right-hand side $f : [0, T] \times \mathbb{R}^m \to \mathbb{R}^m$ is globally Lipschitz continuous with respect to a norm $\|\cdot\|$ on $\mathbb{R}^m$, with Lipschitz constant $L > 0$. For convenience, we only consider the first step, that is, we take $n = 1$.

**(a)** Verify that the nonlinear system of equations can be written as a fixed-point problem $v = G(v)$ in $\mathcal{D} = (\mathbb{R}^m)^s$, where $v \in \mathcal{D}$ is obtained by 'stacking' all intermediate vectors $k_i \in \mathbb{R}^m, i = 1, 2, \ldots, s$, and where $G : \mathcal{D} \to \mathcal{D}$ is defined as

$$v = \begin{pmatrix} k_1 \\ k_2 \\ \vdots \\ k_s \end{pmatrix}, \quad G(v) = \begin{pmatrix} G_1(v) \\ G_2(v) \\ \vdots \\ G_s(v) \end{pmatrix}, \quad G_i(v) = u_0 + \tau \sum_{j=1}^{s} a_{ij} f(c_j \tau, k_j).$$

**(b)** We use the given norm $\|\cdot\|$ on $\mathbb{R}^m$ to define a norm $\|\cdot\|_{\mathcal{D}}$ on $\mathcal{D} = (\mathbb{R}^m)^s$,

$$\|v\|_{\mathcal{D}} = \max_{1 \le i \le s} \|k_i\|.$$

Show that $G$ satisfies a Lipschitz condition on $\mathcal{D}$ with respect to $\|\cdot\|_{\mathcal{D}}$ with Lipschitz constant $\theta = \alpha L \tau$, where $\alpha = \max_i \sum_j |a_{ij}|$.

**(c)** Use the results from the previous task to prove that, if $\alpha L \tau < 1$, then $G$ has a unique fixed point $v^*$ in $\mathcal{D}$.